

实 验 报 告

实验名称: 实验六 TopoICA 模型实现

课程名称: 认知科学与类脑计算

实验地点: K2 227

实验日期: 2025/9/12

班 级: 23 级智能班

姓 名: 宋浩宇

学 号: 202300130183

一 实验目的：

加深对 TopoICA 模型的理解，能够使用 TopoICA 模型解决简单问题

二 实验环境：

主系统：Windows 11 家庭中文版 23H2 ; ArchLinux

编辑器：Visual Studio Code ; PyCharm 2025.1.2 ; neovim 0.11

Python 解释器：Python 3.9

Pytorch 版本：2.8.0

CUDA 版本：12.8.97

硬件环境：

CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz

内存：32.0 GB (31.6 GB 可用)

磁盘驱动器：NVMe WD_BLACKSN850X2000GB

显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU

CUDA 核心数：7424

三 实验内容：

根据 TopoICA 模型的相关知识，使用 Python 语言实现一个简单的 TopoICA 模型。

四 实验步骤：

1. 实验程序：

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_data =
datasets.MNIST(root='F:\\DATASET\\MNIST\\mnist_dataset_dir',
train=True, transform=transform, download=True)
test_data =
datasets.MNIST(root='F:\\DATASET\\MNIST\\mnist_dataset_dir',
train=False, transform=transform, download=True)
train_loader = torch.utils.data.DataLoader(train_data,
batch_size=256, shuffle=True)
```

```

test_loader = torch.utils.data.DataLoader(test_data, batch_size=256,
shuffle=False)
class TopoICA(nn.Module):
    def __init__(self, input_dim=784, n_components=64, n_classes=10):
        super(TopoICA, self).__init__()
        self.W = nn.Parameter(torch.randn(input_dim, n_components) *
0.01)
        self.classifier = nn.Linear(n_components, n_classes)
    def forward(self, x):
        s = torch.matmul(x, self.W)
        logits = self.classifier(s)
        return s, logits
    def reconstruct(self, s):
        return torch.matmul(s, self.W.T)
def topo_loss(W, lambda_topo=0.1):
    diff = W[1:] - W[:-1]
    return lambda_topo * torch.mean(diff ** 2)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = TopoICA().to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion_cls = nn.CrossEntropyLoss()
def ica_loss(x, x_hat):
    recon = torch.mean((x - x_hat) ** 2)
    sparsity = torch.mean(torch.abs(x_hat))
    return recon + 0.01 * sparsity
num_epochs = 5
for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for images, labels in train_loader:
        images = images.view(images.size(0), -1).to(device)
        labels = labels.to(device)
        s, logits = model(images)
        x_hat = model.reconstruct(s)
        loss_unsup = ica_loss(images, x_hat) + topo_loss(model.W)
        loss_sup = criterion_cls(logits, labels)
        loss = loss_unsup + 0.1 * loss_sup
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch [{epoch+1}/{num_epochs}], Loss:
{total_loss/len(train_loader):.4f}")
model.eval()

```

```

correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images = images.view(images.size(0), -1).to(device)
        labels = labels.to(device)
        _, logits = model(images)
        pred = torch.argmax(logits, dim=1)
        correct += (pred == labels).sum().item()
        total += labels.size(0)
acc = correct / total
print(f"Test Accuracy: {acc:.4f}")
W = model.W.detach().cpu().T
fig, axes = plt.subplots(8, 8, figsize=(8, 8))
for i, ax in enumerate(axes.flat):
    ax.imshow(W[i].reshape(28, 28), cmap='gray')
    ax.axis('off')
plt.suptitle("Learned Independent Components (TopoICA)")
plt.show()

```

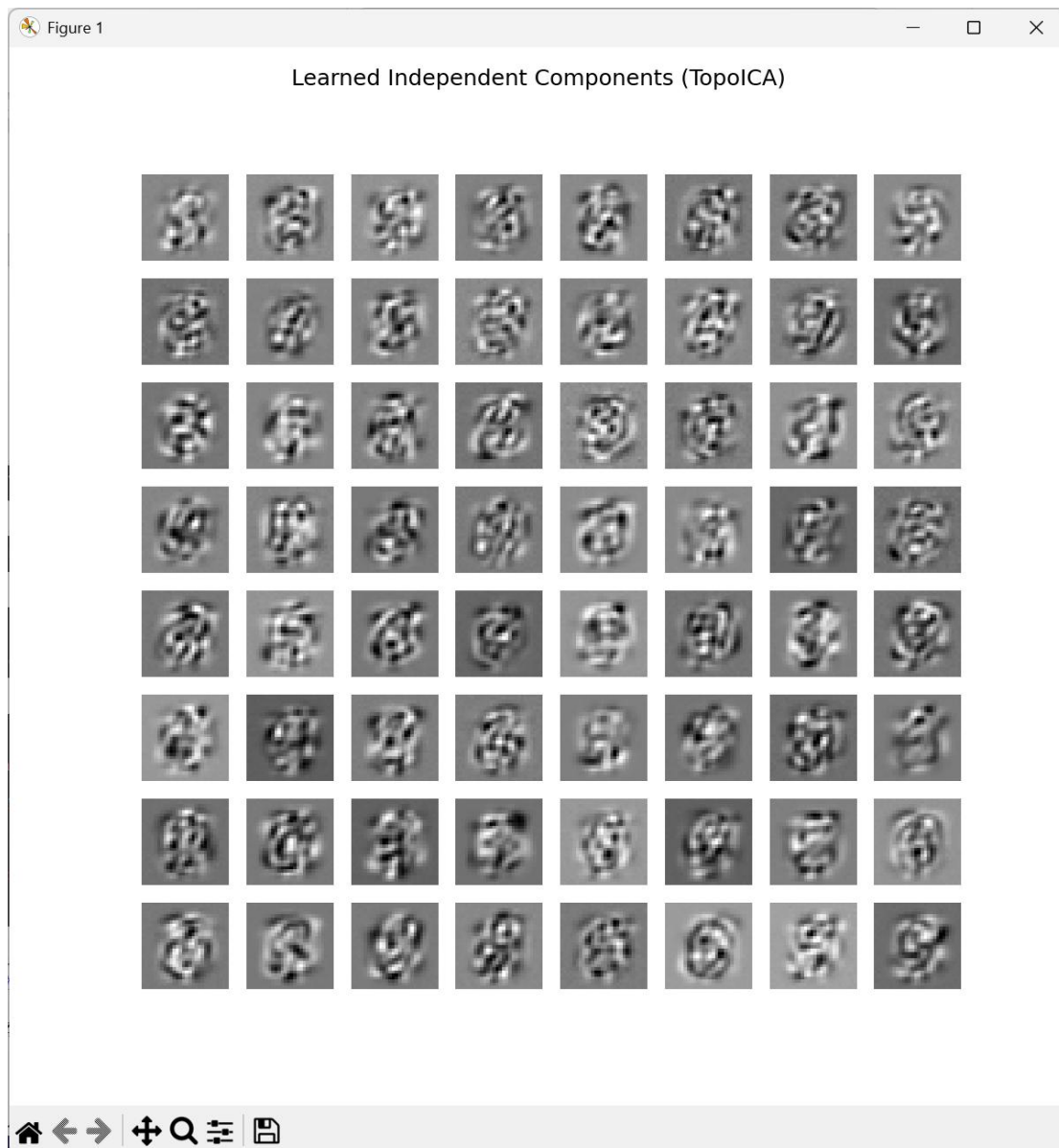
2. 程序实验结果:

```

F:\Homework\认知科学与类脑计算作业\.venv\Scripts\python.exe F:\Homework\认知科学与类脑计算作业\实验6\TopoICA.py
Epoch [1/5], Loss: 0.2332
Epoch [2/5], Loss: 0.1212
Epoch [3/5], Loss: 0.0994
Epoch [4/5], Loss: 0.0903
Epoch [5/5], Loss: 0.0860
Test Accuracy: 0.9038

进程已结束，退出代码为 0

```



3. 分析和改进:

从结果上来看，模型的准确度还算可以，能够成功解决这个问题。从可视化结果可以看到，模型学习到的独立成分多呈现出数字笔画、边缘等局部特征结构，表明拓扑正则项在特征平滑与空间结构保持方面起到了一定作用。改进的话可以考虑引入非线性特征映射，因为目前模型仅使用线性变换，可在前端加入浅层卷积或非线性激活以提升特征表达能力。

五 实验小结:

本实验通过在 MNIST 数据集上实现并训练一个 TopoICA 模型，验证了结合独立成分分析与拓扑约束的思路在图像特征学习中的可行性。实验结果显示，模型不仅能够学习到有意义的结构化特征，还具备一定的分类性能。依旧还拥有可以改进的地方。